

SMMA — Organização das pastas

Acelerador FPGA para manutenção preditiva · DE0-CV (Cyclone V, 50 MHz) · 151 arquivos no projeto · 09/10/2026

Visão geral

SMMA/

├─ RTL/

hardware em Verilog – 48 arquivos, uma pasta por bloco do diagrama

├─ top/

entrada/ lms/ barramento/ memorias/ fft/

├─ mdc/

matriz/ ml/ cnn/ aritmetica/

├─ sim/

simulação

├─ tb/

39 testbenches, mesmas pastas do RTL

├─ golden/

modelos de referência (FFT, CNN)

├─ modelsim/

scripts .do

├─ run_regressao.sh · run_xcelium.sh · filelist.f

├─ quartus/

projeto do Quartus – abrir SMMA.qpf

├─ SMMA.qpf · SMMA.qsf · SMMA.sdc

├─ vetores/

.hex das ROMs (\$readmemh) e vetores de teste

├─ python/

treino, quantização e geração das ROMs

├─ smma/

scripts/ resultados/

├─ docs/

documentação, enunciado e diagramas

├─ README.md · LICENSE · .gitignore

├─ dados/

dataset (só no seu computador, fora do git)

Como as pastas se ligam

Placa (Quartus)

quartus/SMMA.qsf lista os 48 arquivos de RTL/ pelo caminho ../RTL/<pasta>/. As ROMs abrem vetores/*.hex, por isso vetores/ fica ao lado do .qpf.

Simulação

sim/run_regressao.sh compila todo o RTL/ com cada testbench de sim/tb/, rodando numa cópia de quartus/vetores/. Resultado atual: 39/39.

Python → hardware

Os scripts de python/scripts/ geram os .hex de quartus/vetores/ e a RTL/cnn/CNN_Weight_ROM.v. Não é preciso rodá-los para usar a placa.

Fora do git

dados/ (dataset) e as pastas que o Quartus gera (db/, incremental_db/, output_files/ com o SMMA.sof) ficam só no seu computador.

RTL/ — hardware 48 arquivos · pasta = bloco do diagrama do grupo

RTL/top/	SMMA_Top · CONTROLE GLOBAL · SAÍDA
Top level, controle global e saída · 3 arquivo(s)	
SMMA_Global_Control.v	controle global: start unico, espera todos os blocos, veredito
SMMA_Panel.v	saída: classes nos displays HEX e LEDs da DE0-CV
SMMA_Top.v	top level do SMMA na DE0-CV (diagrama em docs/diagramas)
RTL/entrada/	SENSOR Xa · FIR + /8
Interface de entrada dos sensores · 2 arquivo(s)	
FIR_Decimator.v	FIR anti-alias de 63 taps + decimacao por 8 (25,6 → 3,2 kHz)
Sample_Source.v	SENSOR Xa: ROM com 12 janelas do dataset, entregues a 25,6 kHz

RTL/lms/		LMS
Filtro adaptativo LMS (enunciado 3.4) · 7 arquivo(s)		
LMS_Accumulator_Error_Scale.v	acumula $y(n)$, calcula $e(n)$ e o passo $\mu \cdot e(n)$ do LMS	
LMS_Control_FSM.v	escalador do LMS de 8 taps com recursos compartilhados	
LMS_Filter_Top.v	filtro LMS adaptativo de 8 coeficientes (enunciado 3.4)	
LMS_Input_Delay_Line.v	linha de atraso $x(n-1) \dots x(n-8)$ do LMS	
LMS_Processing_Element.v	elemento de processamento do LMS (MAC + atualizacao de peso)	
LMS_Stage.v	LMS em serie na entrada: repassa as amostras e calcula r_{lms}	
LMS_Weight_Storage.v	banco dos 8 pesos do LMS em Q1.15	
RTL/barramento/		DATA BUS DRIVER
Comunicação entre módulos · 2 arquivo(s)		
Data_Bus_Driver.v	entrega as amostras filtradas aos dois ramos	
Stream_Fork.v	replica um stream valid/ready para N consumidores (join)	
RTL/memorias/		MEM_A · MEM_B
Memórias de amostras e de espectros · 2 arquivo(s)		
Frame_Builder.v	quadros de 64 amostras com salto de 32 para a FFT	
Spectrum_Accumulator.v	acumula $ X[k] $ dos 32 quadros e entrega o espectro medio	
RTL/fft/		FFT
FFT de 64 pontos (enunciado 3.2) · 8 arquivo(s)		
FFT_Addr_Gen.v	enderecos (p, q, k) de cada butterfly da FFT	
FFT_Bit_Reverse.v	inversao de bits do endereco de carga da FFT	
FFT_Butterfly.v	butterfly radix-2 DIT pipelinado, escala 1/2 por estagio	
FFT_Control_FSM.v	maquina de estados da FFT (carga, 6 estagios, saida)	
FFT_Magnitude.v	$ X[k] $ por alpha-max plus beta-min, sem multiplicador	
FFT_Memory.v	RAM dual-port 64×32 bits da FFT (in-place)	
FFT_Top.v	FFT de 64 pontos, radix-2 DIT, Q1.15 (enunciado 3.2)	
FFT_Twiddle_ROM.v	fatores de rotacao W_{64}^k em Q1.15	
RTL/mdc/		PEAK DETECTOR · EUCLIDES · f0
Frequência fundamental (enunciado 3.1) · 3 arquivo(s)		
f0_estimator.v	frequencia fundamental $f_0 = k_0 \cdot f_s / N$ (enunciado 3.1)	
mdc_gcd.v	MDC dos indices dos picos (enunciado 3.1)	
peak_detector.v	os 3 maiores picos do espectro	
RTL/matriz/		COEFFICIENT ACC. · GAUSS-JORDAN
Inversão de matriz (enunciado 3.3) · 4 arquivo(s)		
Yule_Walker_Solver.v	monta a Toeplitz, inverte e calcula $a_1 \dots a_3$ (enunciado 3.3)	
autocorrelacao_yw.v	autocorrelacao $\rho[0..3]$ da janela	
fixed_point_divider.v	divisor sequencial em ponto fixo (1/pivo do Gauss-Jordan)	
gauss_jordan_inv.v	inversao de matriz $N \times N$ ($N \leq 4$) por Gauss-Jordan com pivotamento	

RTL/mL/

BANDAS · PARAMETER REGFILE · DECISION TREE

Acelerador de ML (enunciado 3.5) · 3 arquivo(s)

Feature_Spectral.v	8 características espectrais em Q1.15
ML_Tree_Classifier.v	arvore de decisao em ROM (enunciado 3.5)
Parameter_RegFile.v	vetor de 16 características para a arvore

RTL/cnn/

LOG2 · ESPECTROGR. · CNN

Acelerador CNN (enunciado 3.6) · 11 arquivo(s)

CNN_Control_FSM.v	controle do acelerador CNN (convolucao, pooling, densa)
CNN_Conv_Layer.v	convolucao 3×3, 8 filtros, padding 1, bias e ReLU
CNN_Dense_Classifier.v	global average pooling + camada densa 8×4 + argmax
CNN_Line_Buffer.v	janelas 3×3 por line buffers, com zero-padding
CNN_MAC_Unit.v	multiplica-acumula pipelinado em ponto fixo
CNN_MaxPool.v	max pooling 2×2, stride 2, em streaming
CNN_ReLU.v	requantizacao, saturacao e ReLU na saida dos acumuladores
CNN_Top.v	acelerador CNN: espectrograma 32×32 → 4 scores e classe (enunciado 3.6)
CNN_Weight_ROM.v	pesos treinados da CNN (gerado por python/scripts/05_exportar_rtl.py)
FFT_Log2_Compress.v	log2 (Mitchell) da magnitude → pixel do espectrograma
Spectrogram_Buffer.v	espectrograma 32×32 (32 quadros da FFT) entregue a CNN

RTL/aritmetica/

— (usada pelos blocos acima)

Aritmética de ponto fixo (enunciado 6.4) · 3 arquivo(s)

Divider_Q15.v	divisor por restauracao: $q = (\text{num} \ll 15) / \text{den}$, saturado em Q1.15
FP_Arith_Unit.v	soma/subtracao parametrizavel em ponto fixo
FP_Mult_Unit.v	multiplicador com sinal em ponto fixo (infere DSP)

sim/ — simulação 39 testbenches + scripts

sim/tb/top/	tb_Equivalencia, tb_SMMA_Top, tb_latencia
sim/tb/entrada/	tb_FIR_Decimator, tb_Sample_Source
sim/tb/lms/	tb_LMS_Accumulator_Error_Scale, tb_LMS_Control_FSM, tb_LMS_Filter_Top, tb_LMS_Input_Delay_Line, tb_LMS_Processing_Element, tb_LMS_Stage, tb_LMS_Weight_Storage
sim/tb/barramento/	tb_Stream_Fork
sim/tb/memorias/	tb_Frame_Builder
sim/tb/fft/	tb_FFT_Top
sim/tb/mdc/	tb_MDC_Chain, tb_f0_estimator, tb_mdc_gcd, tb_peak_detector
sim/tb/matriz/	tb_Yule_Walker_Solver, tb_autocorrelacao_yw, tb_gauss_jordan_inv
sim/tb/mL/	tb_Feature_Spectral, tb_ML_Tree_Classifier, tb_Parameter_RegFile

sim/tb/cnn/	tb_CNN_Control_FSM, tb_CNN_Conv_Layer, tb_CNN_Dense_Classifier, tb_CNN_Line_Buffer, tb_CNN_MAC_Unit, tb_CNN_MaxPool, tb_CNN_ReLU, tb_CNN_Top, tb_CNN_Weight_ROM, tb_FFT_Log2_Compress, tb_Spectrogram_Buffer
sim/tb/aritmetica/	tb_Divider_Q15, tb_FP_Arith_Unit, tb_FP_Mult_Unit
sim/filelist.f	lista das fontes RTL (usada pelo Xcelium)
sim/golden/golden_model_cnn.py	modelo de referência bit-exato da CNN (pesos gerados pelo passo 05)
sim/golden/golden_model_fft.py	modelo de referência bit-exato da FFT
sim/modelsim/sim_latencia.do	latência de cada bloco, com ondas
sim/modelsim/sim_tb.do	roda um testbench no ModelSim/Quarta
sim/run_regressao.sh	roda todos os testbenches (Icarus Verilog)
sim/run_xcelium.sh	simulação no Cadence Xcelium (laboratório)

quartus/ — projeto da placa

quartus/SMMA.qpf	projeto do Quartus — é este que se abre
quartus/SMMA.qsf	dispositivo, top SMMA_Top, os 48 arquivos de RTL/ e a pinagem da DE0-CV
quartus/SMMA.sdc	relógio de 50 MHz e caminhos assíncronos (botões, LEDs)

quartus/vetores/ — conteúdo das ROMs (lido na síntese) e vetores dos testbenches:

ARQUIVO	USO	CONTEÚDO	LIDO POR
arvore.hex	síntese	141 nós da árvore de decisão	ML_Tree_Classifier
demo_amstras.hex	síntese	12 janelas do dataset (8503 amostras cada)	Sample_Source
demo_rotulos.hex	síntese	classe verdadeira e prevista de cada janela	Sample_Source
fir_coef.hex	síntese	63 coeficientes do FIR	FIR_Decimator
clf_teste.hex	teste	64 vetores + classe do sklearn	tb_ML_Tree_Classifier
conv_esperado.hex	teste	saída esperada da convolução	tb_CNN_Conv_Layer
demo_info.txt	informativo	de qual arquivo do dataset veio cada janela	—
dense_esperado.hex	teste	saída esperada da densa	tb_CNN_Dense_Classifier
equiv_esperado.hex	teste	saídas da integração gravada na placa	tb_Equivalencia
feat_teste.hex	teste	espectros médios + 8 características esperadas	tb_Feature_Spectral
fir_teste.hex	teste	entrada e saída esperada do FIR	tb_FIR_Decimator
rom_pesos.hex	teste	os 116 pesos da CNN	tb_CNN_Weight_ROM
temp_teste.hex	teste	6 janelas decimadas + r_lms e rho esperados	tb_LMS_Stage, tb_autocorrelacao_yw, tb_Yule_Walker_Solver
top_esperado.hex	teste	scores e classe esperados	tb_CNN_Top
top_imagens.hex	teste	8 espectrogramas reais	tb_CNN_Top
top_origem.txt	informativo	origem das 8 imagens	—

python/ — treino e geração das ROMs

python/README.md	passos do treino, dataset, resultados da CNN e da árvore
python/config.py	parâmetros e caminhos do fluxo
python/requirements.txt	dependências (numpy, torch, sklearn)
python/smmap/	espectrograma.py · features.py · modelo.py · golden.py — bibliotecas do fluxo (espelham o hardware bit a bit)
python/scripts/	passos 01 a 11: dataset → espectrogramas → treino → avaliação → ROMs e vetores de teste
python/resultados/	pesos da CNN de 16 bits (modelo_16b.pt, pesos_16b.npz), árvore (arvore_float.npz, comparacao_clf.npz), relatórios gerados e matriz_confusao.png

docs/ e raiz

docs/ARQUITETURA.md	arquitetura final, relação dos módulos, formatos, latências e recursos
docs/PBL_enunciado.pdf	enunciado do PBL
docs/QUARTUS_E_PLACA.md	compilar, gravar e usar na DE0-CV; resultado esperado
docs/SMMA_casos_de_teste_placa.pdf	ficha de testes na placa (esperado × obtido)
docs/SMMA_organizacao_pastas.pdf	este documento
docs/VERIFICACAO.md	os 39 testbenches e a ordem de teste
docs/diagramas/SMMA_arquitetura.pdf	o mesmo desenho, para impressão
docs/diagramas/SMMA_arquitetura.png	desenho da arquitetura final
README.md	visão geral, mapa das pastas e links
LICENSE	licença
.gitignore	o que não vai para o git (dataset, saídas do Quartus e dos simuladores)

O que saiu nesta organização

- **Comentários:** cada arquivo Verilog ficou com um cabeçalho de uma linha e só os comentários curtos. Os comentários caíram de 3 869 para 1 080 linhas. As explicações longas foram para docs/ARQUITETURA.md.
- **Código:** nenhuma alteração. Os 87 arquivos .v foram comparados token a token com a versão gravada na placa e são idênticos; a regressão continua em 39/39.
- **Pastas:**
 - buffers/ virou memorias/ (MEM_A e MEM_B);
 - FFT_Log2_Compress e Spectrogram_Buffer foram para cnn/, que é o ramo CNN do diagrama;
 - Data_Bus_Driver e Stream_Fork foram para barramento/;
 - comum/ virou aritmetica/.
- **Removidos (sem uso):**
 - figuras PNG dos espectrogramas de teste e o script que as gerava;
 - pesos experimentais de 4 e 8 bits, cujos resultados estão em python/README.md;
 - logs de treino;
 - a versão SVG do diagrama;
 - o PDF de latência da integração antiga.
- **Documentos:** ficaram cinco: o README.md da raiz, três em docs/ (arquitetura, Quartus/placa, verificação) e o python/README.md, que juntou o guia da CNN e a análise do treino.